Introduction to Observability

chukmunnlee@nus.edu.sg

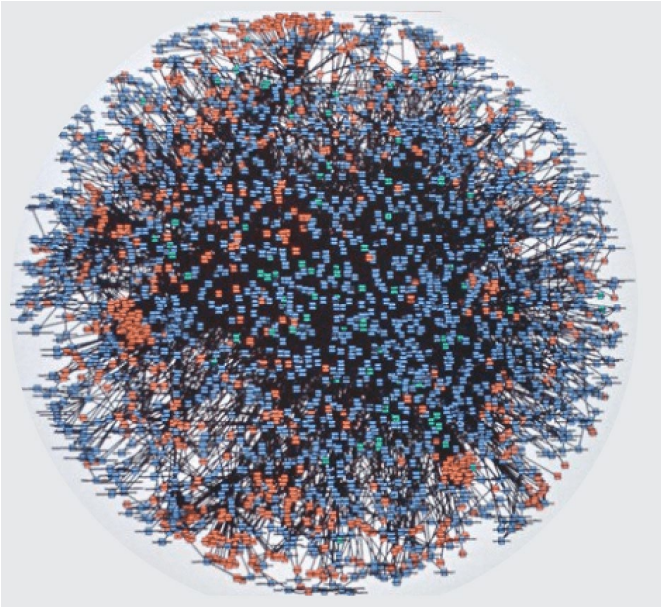
© Copyright National University of Singapore

What is Observability?

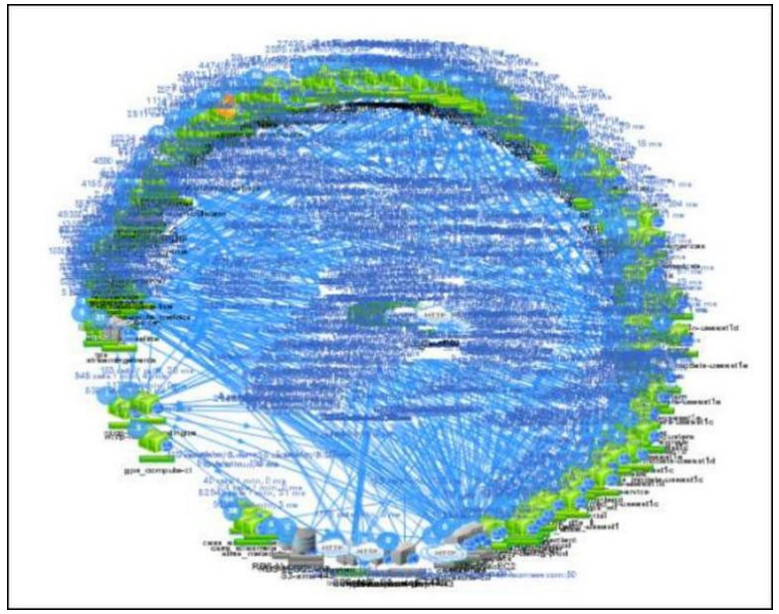
- Understand the internal system state by measuring externally available data
  - Originated from control theory
  - Assume that system is continuously generating and emitting data
- Use observation to discover actionable insights
  - Eg. why is database query slow in spite low CPU utilization?
  - Answers the question of what is happening with my system
- Continuous feedback loop
  - Emits the necessary data to understand system behaviour
  - Use data to make improvements to the system

© Copyright National University of Singapore

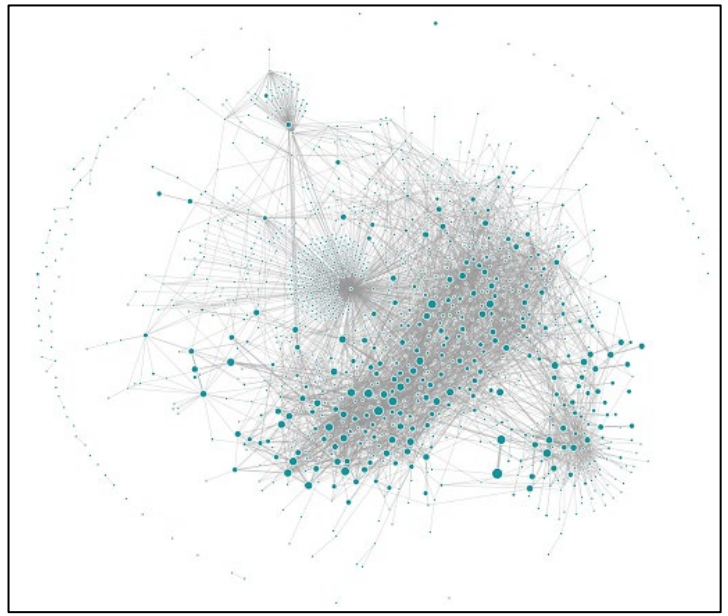
Why Do We Need Observability?



AWS circa 2008



Netflix circa 2018

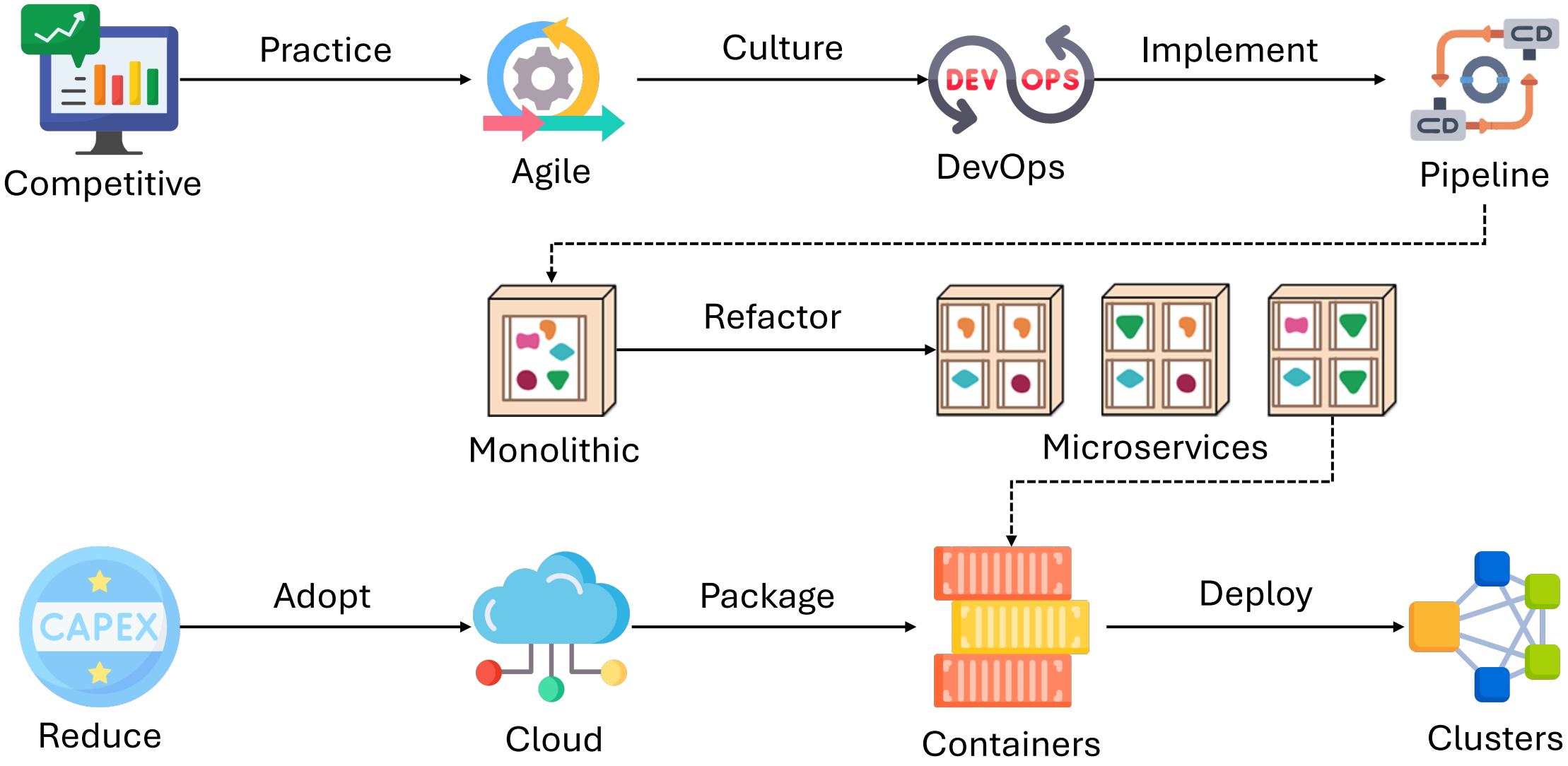


Uber circa 2019

* We're not all AWS, Netflix and Uber

© Copyright National University of Singapore

How Did We Get Here?



Why is Observability Important?

- Provides a holistic view of the systems internal state
  - Proactively detect issues by observing and analyzing these data
  - Identify bottlenecks leading to improved performance
- Modern cloud native applications require different approach to 'monitoring'
  - Multiple layers, involving many moving parts
    - Infrastructure, networks, micro services
  - Microservices has complex interactions where failures may not be reproducible
  - Fast pace of release because of agile and DevOps practices
- Facilitates data-driven decision making
  - Use collected telemetry data for capacity planning

© Copyright National University of Singapore

Why is Observability Important?

- Improve resiliency
  - Providing continuous feedback loop from chaos engineering experiments
  - Insights into how the system behaves when (partial) failure occurs
- Business impact
  - Faster mean time to detection (MTTD), potentially can reduce the mean time to resolution (MTTR)
  - Better resource utilization by identifying the 'hot spots'
- Answering questions
  - The checkout is slow, but I do not know which service is the bottleneck.
  - Users are reporting errors, but the application logs are not showing any errors

© Copyright National University of Singapore

Difference Observability and Monitoring

Monitoring

- Track known metrics
- Focus on generating alerts based on specific symptoms
- Relies on a limited set of predefined metrics
- Reacting to faults
- Answers the question "Is something wrong?"

Observability

- Investigate unknown issues
- Focus on understanding the system from its telemetry data
- Uses and correlates all signals
- Proactively responding to immanent faults
- Answers the question "Why is something wrong?"

© Copyright National University of Singapore

Terms

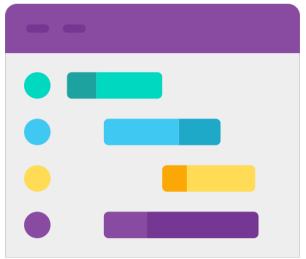
- Telemetry data
  - Information about the internal state of application/infrastructure
  - AKA signals - metrics, logs and traces
- Instrumented application/infrastructure
  - System that is emitting telemetry data
  - Types of instrumentation
    - auto - automatically adding instrumentation code to an application during runtime
    - manual - programmatically add code to emit telemetry data
- Collector
  - Collects telemetry data for processing, exporting or storage
- Observability
  - Act or process of analysing the telemetry data to understand or infer the internal states of a system

© Copyright National University of Singapore

Signals

- Metrics
  - Numerical representation of the system's state over time
  - Aggregated overtime to identify behaviours and anomalies
  - Eg. request latency, request count, CPU utilization

- Logs
  - Discrete timestamped records of events, with associated metadata, that happen at a specific point in time
  - Eg. invalid login, databased connection failed, application exception



- Traces
  - End to end recording of a request flow as it traverses different services
  - Helps to understand an event by providing context and correlation to metrics and logs
  - Eg. access logs, server logs, event logs

© Copyright National University of Singapore

Logs, Metrics and Traces

- View the signals as complementary
- Request-scoped metrics
  - Metrics associated with a request as it travels through the system
  - Aggregated over time to provide insights into the request especially 'important' request like checkout
- Request-scoped events
  - Discrete timestamped events the occurs within the context of a request
- Aggregatable events
  - Events that can be combined and statistically analysed

Image from https://peter.bourgon.org/blog/2017/02/21/metrics-tracing-and-logging.html

© Copyright National University of Singapore

Examples of Metrics, Logs and Traces

```
http_requests_total{method="GET", endpoint="/api/data", status="200"} 12345
```

```
127.0.0.1 - - [05/Aug/2025:13:00:00 +0800] "GET /api/data HTTP/1.1" 200 1024
```

```
{
  "name": "process_request",
  "context": {
    "trace_id": "5b8aa5a2d2c872e8321cf37308d69df2",
    "span_id": "93564f51e1abe1c2"
  },
  "parent_id": "051581bf3cb55c13",
  "start_time": "2025-08-05T13:57:00.114492Z",
  "end_time": "2025-08-05T13:57:00.114631Z",
  "attributes": {
    "http.method": "GET",
    "http.url": "/api/data"
  },
  "events": [{
    "name": "database_query_start",
    "timestamp": "2025-08-05T13:57:00.114561Z",
    "attributes": {
      "query": "SELECT * FROM users"
    }
  }]
}
```

© Copyright National University of Singapore

Log, Metrics and Traces

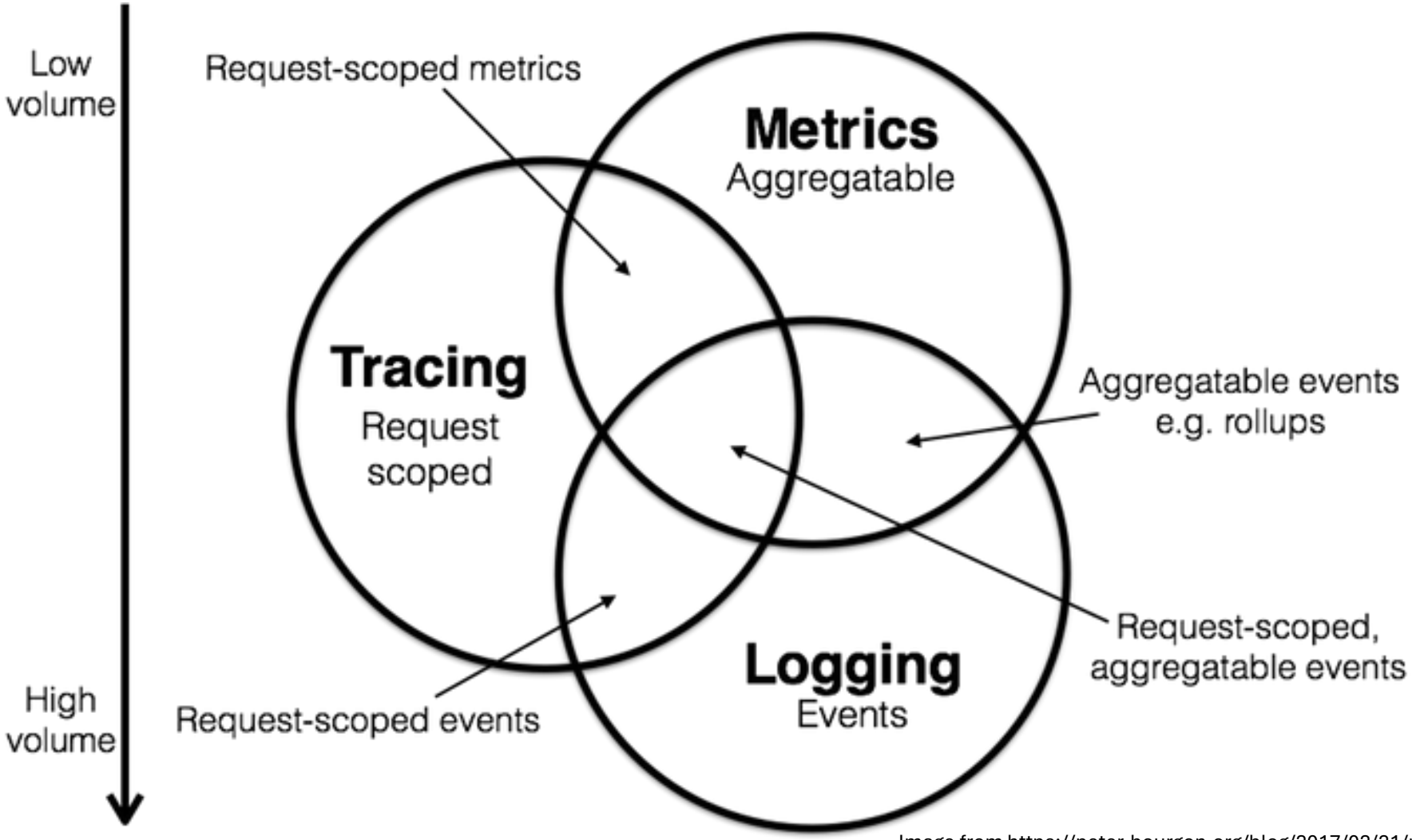


Image from https://peter.bourgon.org/blog/2017/02/21/metrics-tracing-and-logging.html

© Copyright National University of Singapore

Examples of Signal Correlation

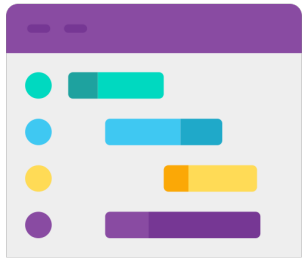
Metrics

Aggregated value representing an event over a period
Eg. 10 packets of chips were purchased in the last 5 minutes

Detailed information about the event
Eg. From time/date, which vending machine, location, payment type

Logs

Interaction that generated the event
Eg. Process of purchasing a bag of chip from a vending machine to payment approval



Traces

© Copyright National University of Singapore

Observability Stack

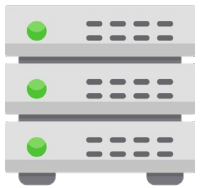
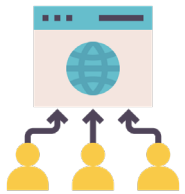
Backend Tools

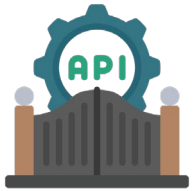
Storage

Processing and Distribution

Collecting Telemetry

Logs, Traces, Metrics



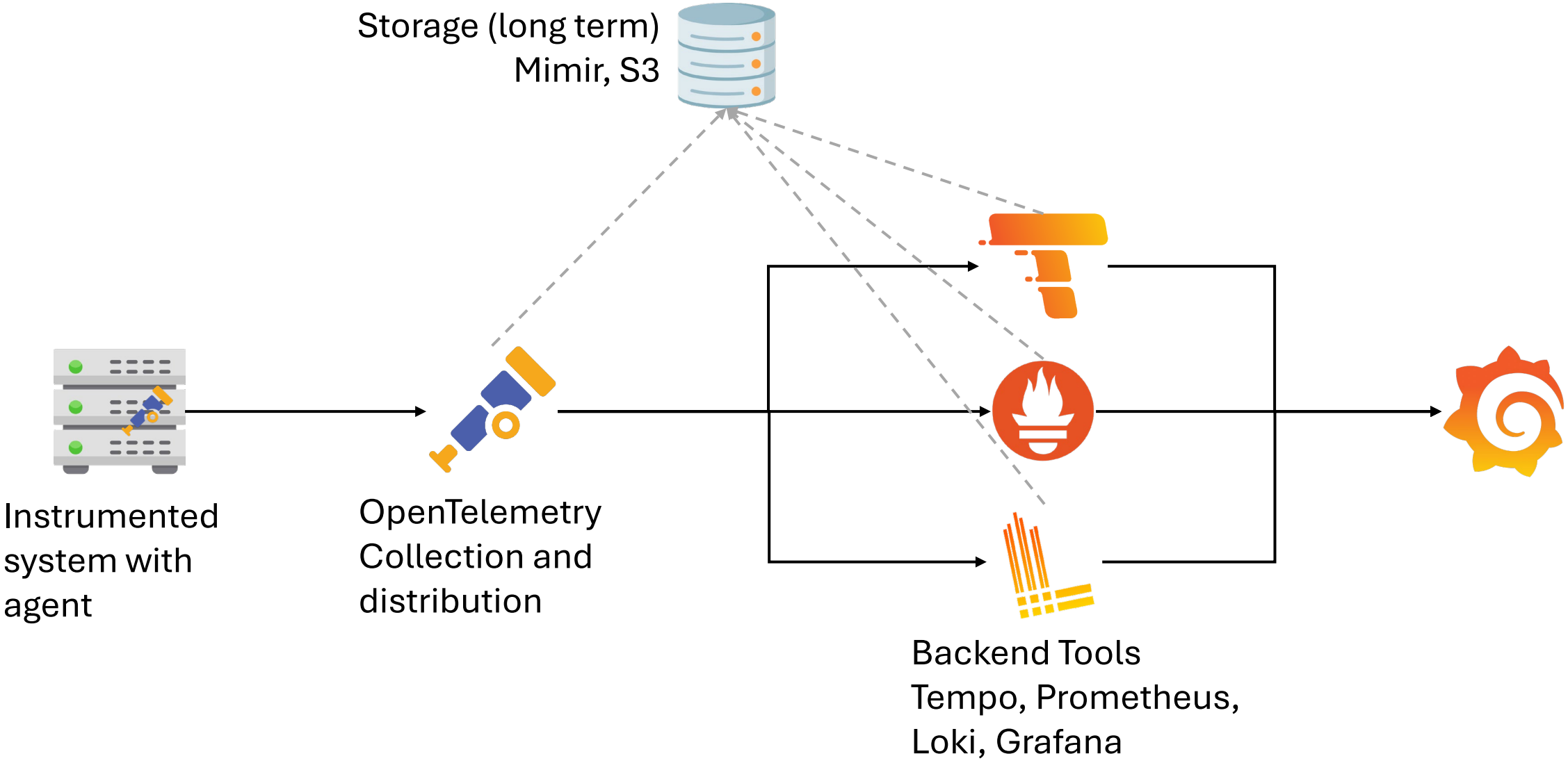
© Copyright National University of Singapore

Observability Stack

- A collection of tools that provides insights into a software system
- Major components
  - Sources - the infrastructure, applications and components that are under observation
    - Eg. applications, API gateway, applications, databases
  - Signals - telemetry data generated by the sources
    - Eg. CPU utilization, error rate, application health
  - Collection - process of collecting the signals
    - Can be performed by an agent running on the server/host
    - Push by the application or pulled by the collector
  - Processing and distribution - collected telemetry data are processed before forwarding
    - Eg. standardized to a specific schema, enriched, sampled
  - Storage - telemetry data are stored either in a cyclic buffer or long term storage
  - Backend tools - tools that offer visualization, querying, analysis and alerting

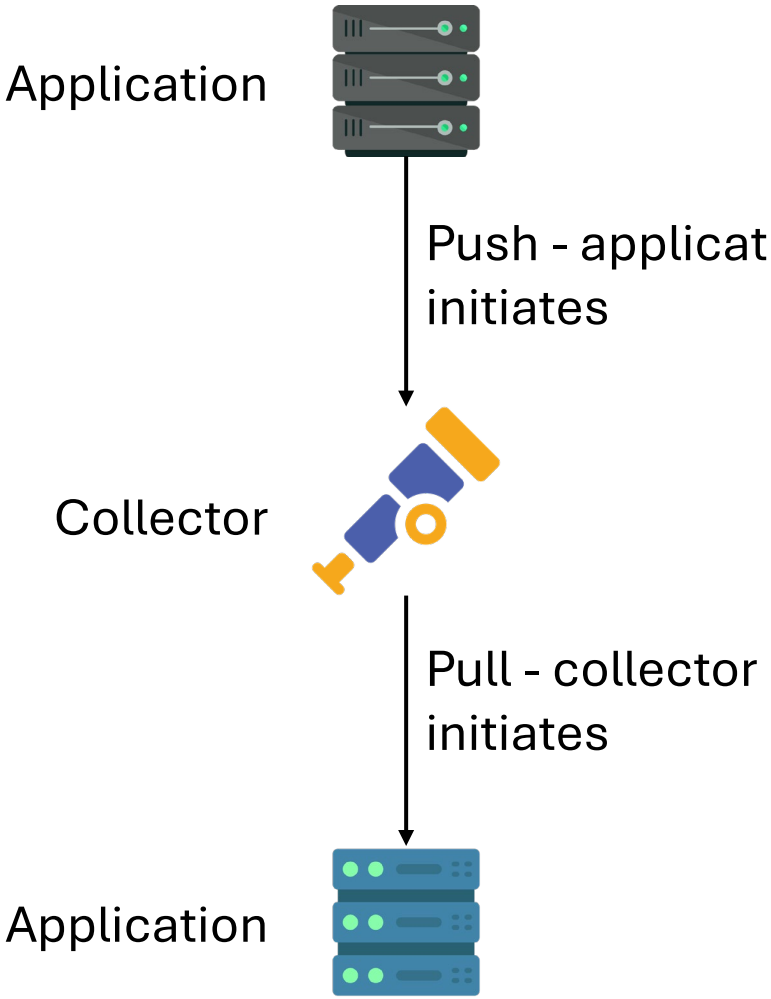
© Copyright National University of Singapore

Example - Observability Stack



© Copyright National University of Singapore

Collecting Telemetry - Push vs Pull



- Push model - application sends telemetry data to the collector
  - Send data whenever its available
  - Can overload the collector when there are lots of application and frequent pushes
  - Collector cannot discover if the application is down when it has not received data
- Pull model - collector request telemetry data from the application
  - Centralized control, especially in dynamic environment where applications can be automatically discovered and scraped
  - Not ideal for short-lived jobs like Cron jobs

© Copyright National University of Singapore

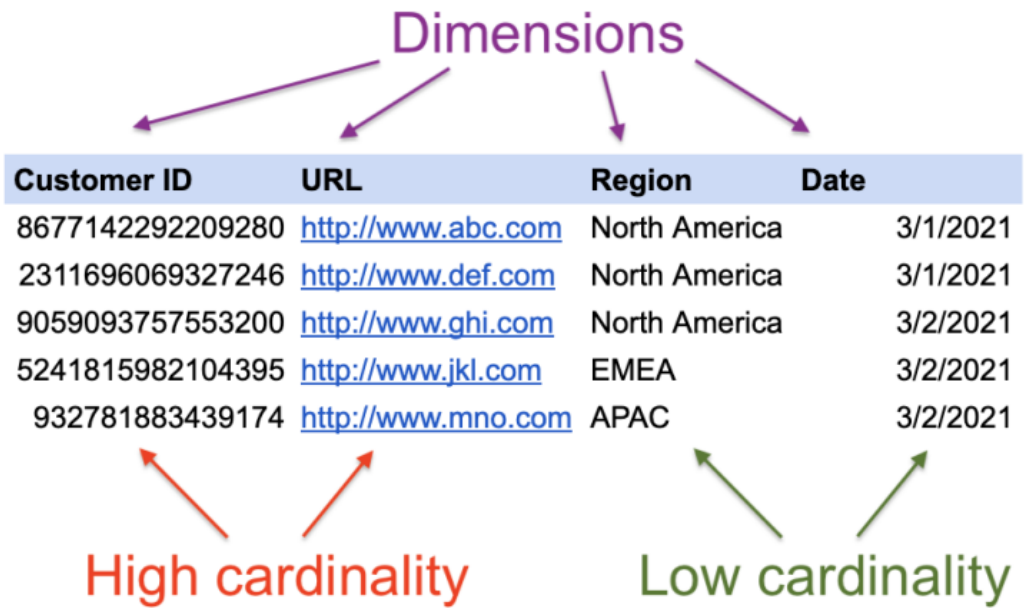
Data Volume

- Telemetry generates large amount of data
  - Logs are typically the largest contributor to this data volume
- Factors that influence volume
  - Scale of the system - the more services we are instrumenting, the data will be generated
  - Traffic volume - application that handles 10K req/sec will generate more telemetry data than an application that handles 1K req/sec
  - Cardinality - number of unique combinations of labels for a given telemetry data especially in timeseries database
    - Each unique combination of creates a timeseries
  - Data level - eg. setting the log level to DEBUG will generate more logs than INFO level

© Copyright National University of Singapore

Dimension and Cardinality

- Dimensionality refers to the number of keys in telemetry data
  - Eg status, level, timestamp, etc
  - High dimension (wide) data captures more details
- Cardinality refers to the number of unique values for a given key
  - Eg. **`region`**, status
  - Used to identify data

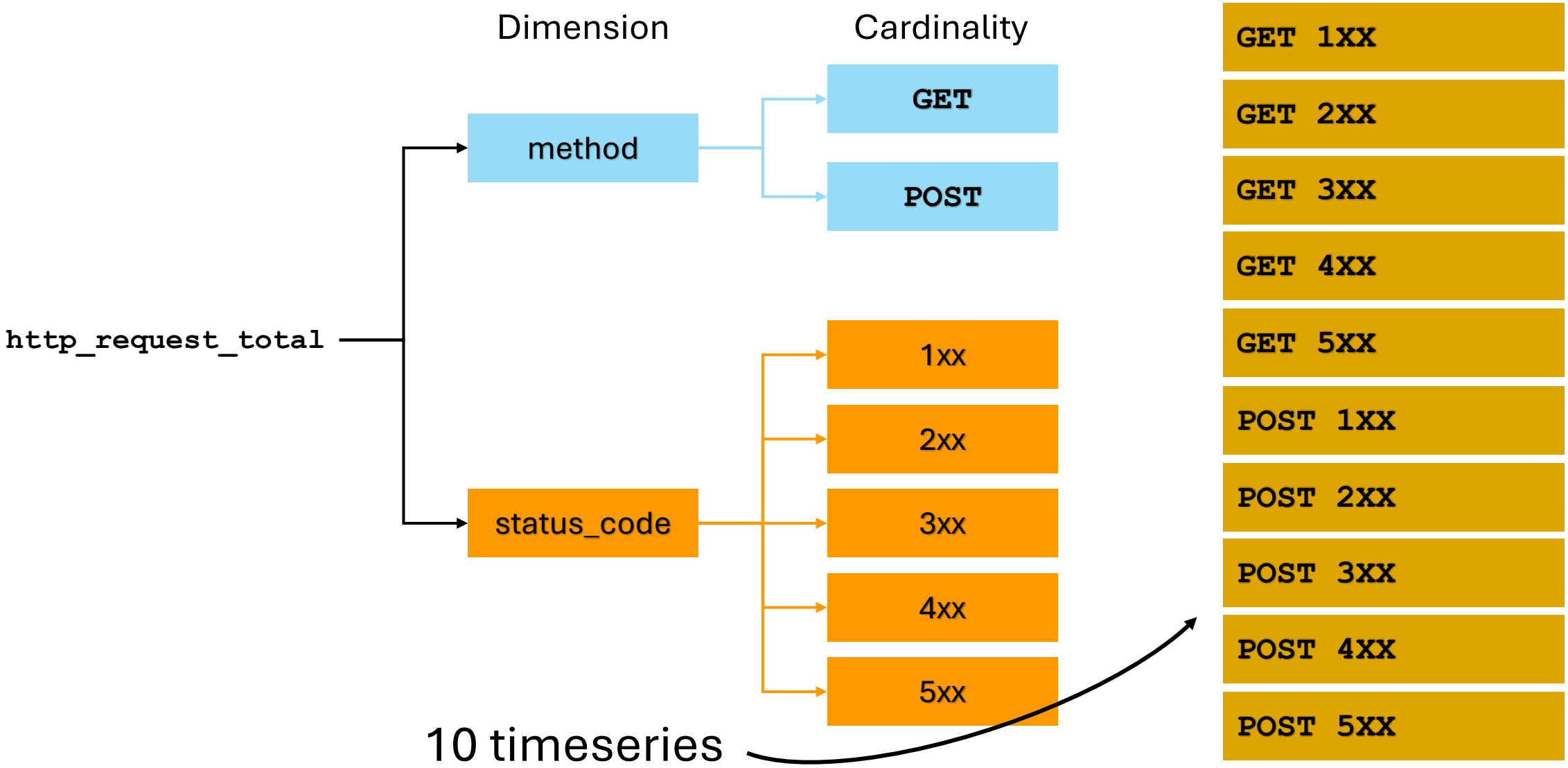


- Impacts performance and data storage

Image from https://newrelic.com/sites/default/files/styles/900w/public/2021-04/High-cardinality-low-cardinality-data.png?itok=dvLJZiiz

© Copyright National University of Singapore

Example - Dimensionality and Cardinality



Adapted from https://grafana.com/blog/2022/02/15/what-are-cardinality-spikes-and-why-do-they-matter/

© Copyright National University of Singapore

Strategies for Managing Data Volume

- Filtering - drop low-value data
  - Eg health check requests from logs, traces and metrics
- Sampling - capture a subset of the data
  - Eg. Head-based and tail-based sampling are often used in traces
- Aggregate high-cardinality data by converting fine-grained to coarse-grained data
  - Eg. Sum failed logins with a single metric **`login_failure_total`** instead of capturing every failed login

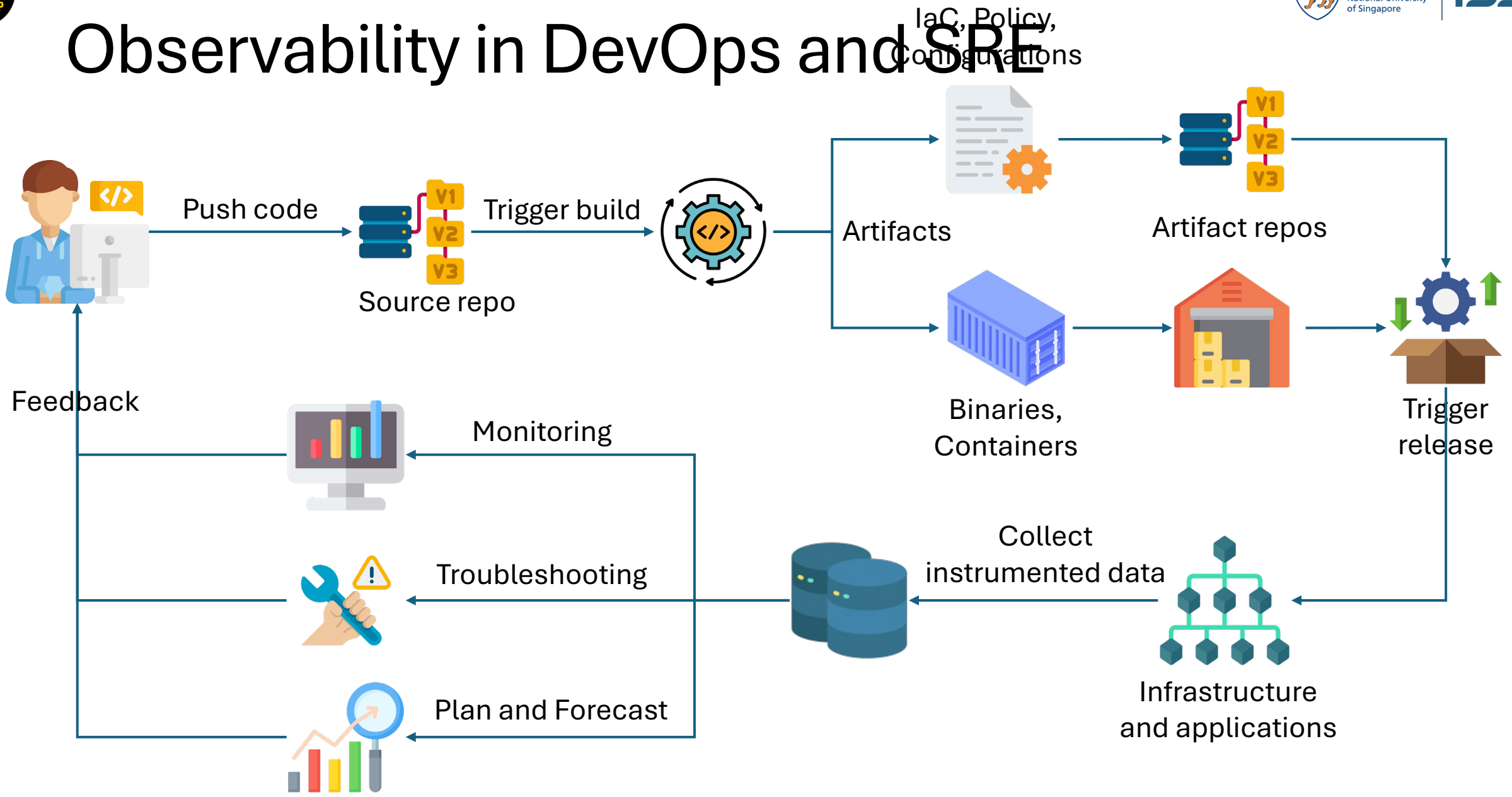
© Copyright National University of Singapore

Data Quality

- Quality of the observability is directly proportional to the quality of the collected signals
  - Collect too much or too little may lead to too much noise or an incomplete view of the system
- Guidelines
  - Relevance - useful to the defined objective
  - Accuracy - at the granularity level to provide the required insights
  - Timeliness - current enough to make useful real-time conclusions
  - Comparability - can the data compared/co-related with other signals
  - Completeness - is the data complete

© Copyright National University of Singapore

Observability in DevOps and SRE



Site Reliability Engineering

- Tracks error budgets
  - SRE engineers rely on SLI to measure performance
  - Alert when error budget is about to be exhausted
    - Error budget: 99.9% / month uptime, then error budget is 0.1% / month
- Shorten feedback loop by providing instant visibility for infrastructure and code changes
  - Use telemetry data to confirm if changes has improved or degraded the system based on a set of defined metrics eg RED
- Improve incident detection and response
  - Surface anomalies (near real-time) before customer complain
  - Enables faster mean time to recovery (MTTR)

© Copyright National University of Singapore

Observability and DevOps

- Observability provides insights and feedback into DevOps practices
  - Continuous feedback loop
    - Observability provides immediate feedback on the system based on quantitative telemetry data
  - Observability tools allows developers to observe their application's behaviour in production
    - Create visualization to show patterns of behaviours
  - Reduce incident response and recovery time
    - Helps operations to detect problems quickly be examining and correlating signals
  - Automation support
    - Observability can provide data needed to trigger automate actions like autos scaling, or acting as gates in CI/CD pipeline

© Copyright National University of Singapore

About This Course

- Focus on observability not monitoring
  - Co-relating signals to answer questions, not create visualization
    - We will create visualization but it is not the main focus
- Will use popular observability tools like Prometheus, Grafana, Loki, OTel Collector, etc
  - Will only teach and use those features to accomplish the objective of the course
  - Feature not related will not be taught eg. deployment, security, relabelling, etc.

© Copyright National University of Singapore

End of Module

© Copyright National University of Singapore